Daily Insurance Fraud Detection Pipeline

Project Documentation — Architecture, Model, and Pipeline Steps

Project Type	Automated Fraud Detection & Alerting
Data Source	Daily Excel exports from the business/claims team
Detection Approach	Rule-based checks + Machine Learning anomaly detection (hybrid)
Notification Channels	Email, Microsoft Teams, Slack
Automation	Fully scheduled — runs daily without manual triggering

1. Project Overview

Insurance claims are submitted by the business team as a daily Excel export. Historically, identifying potentially fraudulent claims required manual review of each day's file — slow, inconsistent, and easy to miss patterns that only become visible across multiple days (for example, the same claimant filing several claims over a week). This project automates that review: every day, the pipeline reads the day's claims, screens them using a combination of business rules and a statistical model, and automatically sends a notification to the business team listing exactly which claims were flagged and why — no manual triggering required.

The design goal was not just detection, but a system the business team can actually trust and act on day-to-day: explainable flags (each one states its reason in plain language), no repeat alerts about a claim already reported, and thresholds that are fully tunable without editing code.

1.1 Why a hybrid approach (rules + ML), not just one or the other

- **Rules alone** catch known fraud patterns reliably and explainably, but only patterns someone has already thought of in advance.
- **ML alone** catches unusual claims a human might not think to write a rule for, but without any labeled fraud history yet, it can't tell you *why* a claim looks unusual — which makes it hard for a business reviewer to trust or act on.
- **Combining both** gives explainable, immediate value from the rules while the ML layer adds a second, independent check — and as confirmed fraud/not-fraud outcomes accumulate month over month, that ML layer can be upgraded to a supervised model trained on real labels.

2. Data Input

Each day, the business team's export is dropped into the pipeline's data folder as an Excel file named with that day's date (e.g. `daily_claims_2026-08-08.xlsx`). The pipeline expects the following columns:

Column	Description
ClaimID	Unique identifier for the claim
PolicyID	Identifier for the insurance policy the claim is filed against
ClaimantName	Name of the person filing the claim
ClaimType	Auto / Health / Property / Life / Travel, etc.
Region	Geographic region of the claim
PolicyStartDate	Date the policy began
ClaimDate	Date the claim was filed
ClaimAmount	Rupee/dollar value of the claim
CoverageLimit	Maximum payout allowed under the policy

Every daily file is kept (not deleted) inside the data folder, because the duplicate-claim and claim-frequency rules only work by comparing a claim against history — a single day's file in isolation isn't enough context to catch those patterns.

3. Detection Logic — Business Rules

Five explainable rules run on every claim. Each one is independently tunable in the configuration file — thresholds can be adjusted by the business team without touching any code.

Rule	Trigger Condition	Rationale
Early Claim	Claim filed within 30 days of the policy's start date	A claim filed almost immediately after coverage begins is a classic fraud indicator (policy taken out specifically to file a claim).
High Amount	Claim amount ≥ 85% of the policy's coverage limit	Claims that push right up against the maximum payout warrant closer scrutiny than typical, moderate claims.
Duplicate Claim	Same policy + same claim amount, filed within 7 days of each other	Catches accidental or deliberate re-submission of essentially the same claim.
High Frequency	Same claimant with 3 or more claims within a trailing 12-month window	An unusually high claim rate from one individual is a well-known fraud signal in the insurance industry.
Data Integrity	Missing key fields, or a claim date that falls before the policy's start date	Catches broken or manipulated records that shouldn't be trusted as-is, separate from fraud judgment.

4. Detection Logic — Machine Learning Model

4.1 Model used: Isolation Forest

The pipeline uses **Isolation Forest** (scikit-learn's `IsolationForest`), an unsupervised anomaly-detection algorithm. It was chosen specifically because it requires **no pre-labeled fraud history** to start working — a critical constraint, since most teams beginning a fraud-detection project don't yet have a dataset of confirmed past fraud cases to train a supervised model on.

4.2 How Isolation Forest works (plain-language)

The algorithm builds many random decision trees that repeatedly split the data on random features and random thresholds. A normal, typical claim tends to require many splits before it sits alone in its own branch — it looks similar to plenty of other claims. A genuinely unusual claim tends to get isolated in very few splits, because there's little else like it nearby. The model scores every claim by how quickly it got isolated, on average, across all the trees — the faster it's isolated, the more anomalous it's considered.

4.3 Features fed into the model

- **ClaimAmount** — the raw claim value

- **days_since_policy_start** — how long after policy inception the claim was filed
- **amount_to_limit_ratio** — claim amount as a proportion of the policy's coverage limit
- **claimant_claim_count** — how many claims this claimant has filed in total

4.4 Key configuration

- **contamination = 0.06** — the model's assumption of what fraction of claims are anomalous (~6%), tunable as real outcomes accumulate
- **n_estimators = 200** — number of random trees built per run, balancing accuracy and speed
- **min_rows_required = 30** — the ML layer is skipped entirely if there's too little data to train on reliably (falls back to rules-only in that case)

4.5 Planned evolution

As the business team confirms which flagged claims were genuinely fraudulent over successive months, those labels can be used to train a **supervised model** (e.g. XGBoost or a logistic regression baseline) in place of Isolation Forest, meaningfully improving precision because the model would then learn from this organization's actual confirmed fraud patterns rather than generic statistical outliers.

5. Scoring and Flagging Logic

Each claim's rule and ML results are combined into a single numeric fraud score, and any claim at or above the configured threshold is flagged for review.

Signal	Points Contributed
Each triggered rule (of the 5 above)	+1 point
ML anomaly flag (Isolation Forest)	+2 points
Flag threshold	Total score ≥ 2 → claim is flagged

This weighting means a single rule alone (e.g. only "early claim") sits just under the threshold unless corroborated by a second rule or the ML layer — reducing false positives from any one rule being overly broad, while two independent signals agreeing is what actually triggers a business-facing alert.

6. Full Pipeline — Step by Step

This is the exact sequence executed on every scheduled daily run.

Step 1 — Trigger

A scheduler (Windows Task Scheduler, cron, or a cloud scheduler) automatically runs the pipeline once per day, with no manual action needed.

Step 2 — Locate the day's file

The pipeline scans the data folder for files matching the daily naming pattern (daily_claims_YYYY-MM-DD.xlsx) and identifies the most recent one, alongside every prior day's file already present.

Step 3 — Load and validate

Every daily file found (today's and all historical ones) is read with pandas and combined into a single dataset. Each file is checked for the required columns before proceeding; a file missing an expected column raises an immediate, clear error rather than silently producing wrong results. Date columns are parsed, and if the same Claim ID appears in more than one file (e.g. a corrected re-send), the most recently loaded version is kept.

Step 4 — Apply the 5 business rules

Each rule (early claim, high amount, duplicate, high frequency, data integrity) runs across the full combined history and attaches a true/false flag plus a plain-language reason to every claim it applies to.

Step 5 — Apply ML anomaly detection

The four engineered features are computed for every claim, and the Isolation Forest model is trained fresh on the day's combined dataset, producing an anomaly flag and score per claim.

Step 6 — Score and flag

Rule and ML results are combined into a total fraud score per the weighting in Section 5; any claim at or above the threshold is marked as flagged.

Step 7 — Filter to only NEW flags

The pipeline loads its persistent state file (a record of every Claim ID already notified on in a previous run) and compares it against today's flagged claims. Only claims never notified before are carried forward to the alert — this is what prevents the same claim from generating a fresh email every single day until it's resolved.

Step 8 — Save the audit report

A full Excel report of every currently flagged claim (new and previously notified alike, each marked accordingly) is saved to the output folder, timestamped with the run date — this is the permanent audit

trail, separate from what gets emailed.

Step 9 — Send notifications

If there are any new flags, an HTML email summary table, a Microsoft Teams adaptive card, and (optionally) a Slack message are generated and sent to the configured recipients / webhooks. If there are zero new flags, no notification is sent that day — avoiding noise.

Step 10 — Update the state file

The Claim IDs just notified on are added to the persistent state file, so tomorrow's run knows not to re-alert on them.

7. Notification Format

Every notification — regardless of channel — includes, per flagged claim: Claim ID, Policy ID, Claimant name, Claim type, Claim amount, the total fraud score, and the specific reason(s) it was flagged (e.g. "Claim filed shortly after policy start; Statistical anomaly (ML)"), so the reviewer never has to guess why a claim appeared.

Channel	Format
Email	HTML table summary, sent via SMTP to a configured recipient list
Microsoft Teams	Adaptive MessageCard with key facts and the top flagged claims, sent via an incoming webhook
Slack	Formatted text message listing flagged claims, sent via an incoming webhook (disabled by default, toggle in config)

A **dry-run mode** is built in and enabled by default: instead of actually sending, the pipeline writes exactly what would have been sent (the email HTML, the Teams JSON payload) to the output folder — letting the pipeline be tested and thresholds tuned with the business team before anything goes out to real inboxes.

8. Automation and Scheduling

The pipeline is designed to run with zero manual triggering. Three scheduling approaches were documented for different environments:

- **Windows Task Scheduler** — a daily trigger runs the Python script at a fixed time each morning, for teams running this on a work PC or Windows server.
- **Linux/Mac cron** — a daily cron entry runs the script for teams on a Linux/Mac server (e.g. `0 7 * * * cd /path/to/fraud_pipeline && python3 src/main.py`).
- **Cloud scheduler** (Power Automate / Azure Function / AWS Lambda / Airflow) — for a fully hands-off setup where the file arrives via SharePoint or email rather than a local folder drop.

Credentials (email password, Teams/Slack webhook URLs) are never stored in the configuration file — they're read from environment variables set on the machine running the schedule, keeping secrets out of the codebase entirely.

9. Configuration and Tunability

Every threshold used across the pipeline lives in a single configuration file, editable by the business team without touching any code: rule thresholds (early-claim window, high-amount ratio, duplicate window, frequency window and count), ML settings (contamination rate, minimum rows required), scoring weights and the flag threshold, and notification settings (channels enabled, recipients, dry-run toggle).

10. Version Control and Deployment

The project is structured as a standard Python repository (config/, data/, src/, README.md, requirements.txt) and tracked in Git, with a .gitignore excluding real claims data, generated reports, and the notification state file from ever being committed — only code, the sample-data generator, and documentation are version-controlled. The repository is hosted on GitHub for backup, collaboration, and change history.

11. Possible Next Steps

- Add a "resolved claims" allow-list so investigated-and-cleared claims stop appearing in the daily audit report entirely, not just stop re-notifying.
- Once several months of confirmed fraud/not-fraud outcomes exist, replace Isolation Forest with a supervised model (e.g. XGBoost) trained on those labels for higher precision.
- As the data folder accumulates months of daily files, migrate from re-reading every Excel file each run to a single running database (SQLite/Parquet) for performance.
- Build a lightweight Power BI or Excel dashboard on top of the accumulating audit reports to show fraud trends over time by region, claim type, and score.